

Structures (struct)

Grouping Related Data for Efficient Programming

1. What is a Structure?

In programming, we often encounter entities that cannot be described by a single number or string. For example, a "Car" isn't just a brand; it's a combination of a brand, a model, a manufacture year, and perhaps its current fuel level.

A **Structure** (or `struct`) allows you to group variables of **different data types** together under a single meaningful name. This turns a set of separate variables into a cohesive object.

Real-World Metaphor: An array is like a egg carton (storing many identical items). A structure is like a Passport (storing different types of info: name, birth date, ID number, and photo).

2. Creating your First Structure

To define a structure, use the `struct` keyword. This is essentially creating your own "Template" for data.

```
struct { // The Template int myNum; // Member: Integer string  
myString; // Member: String } myStructure; // The Variable Instance
```

3. The Dot Operator (.)

Once you have a structure variable, you need a way to reach inside it. In C++, we use the **Dot Operator** to access members.

```
// Assigning values myStructure.myNum = 25; myStructure.myString =  
"Coding is Art"; // Retrieving values cout << myStructure.myNum; //  
Outputs 25 cout << myStructure.myString; // Outputs Coding is Art
```

Syntax Alert: Always remember the semicolon ; after the closing curly brace of a structure definition. Forgetting this is the #1 cause of compilation errors for beginners.

4. Named Structures (The Recommended Way)

The previous example created "Anonymous" structures. In professional C++, we name the structure itself so we can reuse that "type" throughout the program.

Defining a Type:

```
struct Student { string name; int rollNo; float marks; };
```

Creating Multiple Instances:

Now that Student is a recognized type (just like `int` or `bool`), we can create as many as we want:

```
Student s1, s2, s3; s1.name = "Alice"; s2.name = "Bob";
```

This approach is significantly cleaner and allows you to pass these structures into functions or store them in arrays easily.

5. Combining Arrays and Structures

The true power of structures comes when we combine them with other data structures like arrays. There are two ways this happens:

A. Arrays inside Structures

A structure member can itself be an array. This is perfect for storing lists related to an entity.

```
struct Player { string playerName; int recentScores[3]; // An array as a member }; Player p1; p1.playerName = "Hero99"; p1.recentScores[0] = 1500;
```

B. Arrays of Structures

This is extremely common. Imagine a classroom of 50 students. Instead of 50 variables, we create one array of type Student.

```
Student classRoom[50]; classRoom[0].name = "Charlie"; classRoom[1].name = "David";
```

6. Processing with Loops

When you have an array of structures, loops allow you to process massive amounts of data with just a few lines of code.

```
struct Car { string brand; int year; }; Car mallParking[3] =  
{ {"Toyota", 2020}, {"Honda", 2018}, {"Ford", 2022} }; for(int i = 0; i  
< 3; i++) { cout << "Spot " << i << ": " << mallParking[i].brand; cout  
<< " (" << mallParking[i].year << ")" << endl; }
```

Note: You can use standard loops (for, while) to search through structures, for example, finding all cars manufactured after 2021 or summing up all student marks.

7. Comparison: Arrays vs. Structures

Feature	Array	Structure
Data Type	Stores Same types	Stores Different types
Access Method	Via Index [0]	Via Dot Operator .member
Memory	Contiguous block	Grouped block
Real-World Model	A list/sequence	An object/entity

8. Structures vs. Classes

In C++, structures and classes are very similar. The only technical difference is **Access Level**:

- **Struct:** Members are Public by default (anyone can access).
- **Class:** Members are Private by default (restricted access).

Structs are usually used for simple data shipping, while Classes are used for complex logic and Object-Oriented Programming (OOP).

9. Code Challenge Section

Challenge 1: The Inventory Manager

Define a struct `Product` with `name`, `id`, and `price`. Create one instance, take user input for the values, and then print them back in a formatted way.

Challenge 2: Top Student Finder

Create an array of 3 `Student` structures. Use a loop to find the student with the highest marks and print only their name.

```
// Starter Hint: struct Student { string name; float marks; }; Student  
list[3]; // Fill list... find max...
```

10. Summary

- Structures group variables of different types.
- The . operator is used to reach members.
- Named structures are reusable types.
- Arrays of structures are ideal for managing database-like records.
- Structures are the first step toward understanding Classes and OOP.

End of C++ Structures Module